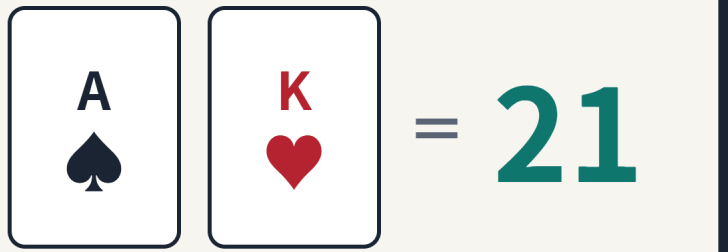
エンジニア研修 ・ オブジェクト指向 開発演習

ブラックジャック演習

まとめて書いてから、
責務を見つけて分ける



責務の分割　カプセル化（不変条件）　多態性

© 2026 by yo1000 | Yoichi KIKUCHI is licensed under CC BY-ND 4.0

01

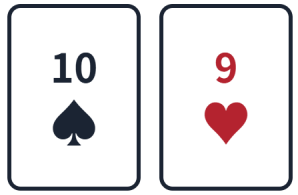
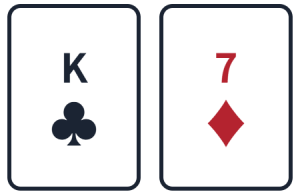
ルール

今回のブラックジャックを知る

ルール

21 に近いほうが勝ち

あなた（プレイヤー）とディーラーの1対1で勝負します。手札の点数の合計が、**21 を超えずに 21 に近いほう**が勝ちです。

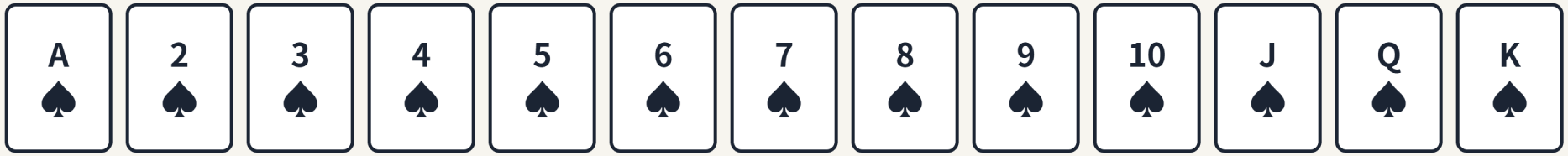
あなた		**19 点** 勝ち
ディーラー		**17 点** 負け

ただし、21 を超えたら（バースト）、その時点で負けです。

ルール

カードの点数

トランプから**ジョーカーを除いた52枚**を使います。スート（マーク）は、点数に関係しません。



カード	点数
2 〜 10	数字のとおり
J・Q・K（絵札）	10 点
A	1 点か 11 点の、都合のよいほう

ルール

ゲームの流れ

1

配る

あなたとディーラーに2枚ずつ配ります。ディーラーの2枚目は伏せておきます。

2

あなたの番

もう1枚引く（Hit）か、やめる（Stand）かを選びます。何度でも引けます。

3

ディーラーの番

伏せたカードを開き、決まりに従ってカードを引きます。

4

勝敗を決める

点数を比べて、勝ち・負け・引き分けを決めます。

21 を超えたら（バースト）、その時点で負けです。

バーストせずに同じ点数なら、引き分け（Push）です。

ルール

1人で遊ぶ

画面にあなたの手札と点数が表示されます。もう1枚引くなら **h**（Hit）、やめるなら **s**（Stand）を入力します。

[h]it

もう1枚引く

[s]tand

ここでやめる

```
You: K of Spades, 5 of Clubs (15)
[h]it or [s]tand? > h
```

ルール

ディーラーは、引くかどうかを選ばない

ディーラーは、点数が 17 点以上になるまで必ず引き、17 点以上になったら必ず止まります。



ディーラーの点数	すること
16 点以下	必ずもう1枚引く
17 点以上	必ず止まる

ルール

勝ち負けの決め方

あなたの番が、ディーラーの番より先に終わります。勝ち負けは、次の順に確かめて決めます。

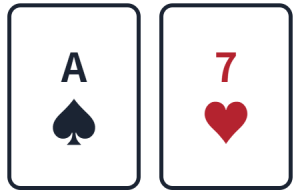
1. あなたがバーストした → その時点で**あなたの負け**。ディーラーはカードを引かない
2. ディーラーがバーストした → **あなたの勝ち**
3. どちらもバーストしていない → 21 に近いほうの勝ち。同じ点数なら**引き分け**（Push）

あなたがバーストした時点で勝負は決まるので、「両方がバースト」にはならない。

ルール ・ 遊び方の例

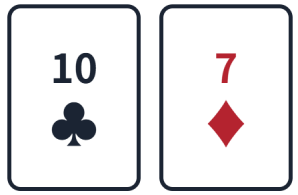
例1：A を 11 点として数える

あなた



18 点 Stand

ディーラー



17 点 止まる

結果：A を 11 点と数えて 18 点。17 点のディーラーに勝ち

ルール ・ 遊び方の例

例2：A を 1 点として数える



結果：A を 11 点のままだと 25 点になるので、A を 1 点にして 15 点。続けて引いて 20 点

ルール ・ 遊び方の例

例3：バーストする



結果：21 を超えたので、ここで負け。ディーラーはカードを引かず、あとから結果が変わることもない

02

進め方

まとめて書いてから、分ける

進め方

今回は、分け方を自分で決める

これまでの演習では、作るクラスが先に決まっていました。今回は、まず動くものを作り、その流れを見ながら分け方を考えます。

これまで

部品から作る

作るクラスと役割が、先に決まっている

作っている間は、全体の流れやクラス同士のつながりが見えにくい

今回

流れから分ける

まず、すべての処理を main に書いて動かす

動く流れを見ながら、どこで分けるかを自分たちで決める

先に全体を動かすと、何と何がつながっているかが見える。

進め方

4つの段階で分ける

一度に分けようとせず、小さな段階を順に進みます。どの段階でも、ゲームの動きは変えません。

1

main に書き下す

クラスもメソッドも作らず、すべてを main に書いて動かす

2

メソッドに切り出す

ひとまとまりの処理に、名前をつけて切り出す

3

クラスに切り出す

いつも一緒に使うデータと処理を、1つにまとめる

4

役割で分ける

種類ごとに違うところを、それぞれに任せる

名前をつける → まとめる → 任せる、の順に、少しずつ形を変えていく。

進め方

切り出す手がかり

コードの中に次のようなところを見つけたら、切り出すきっかけになります。

見つけたもの	切り出し方
同じ処理が、2か所以上に書かれている	メソッドにして、1か所にまとめる
コメントで「〜する」と説明しているかたまり	メソッドにする（コメントが名前の候補）
いつも一緒に受け渡ししているデータ	クラスにまとめる
同じデータを読み書きするメソッドの集まり	データと一緒に、クラスに移す
種類によって、if で書き分けている処理	種類ごとのクラスに任せる（多態性）

進め方

動きを変えずに分ける

切り出すたびに、**同じシードと同じ入力**で動かし、表示が変わらないことを確かめます。切り出す前の表示を before.txt に保存しておきます。

```
./mvnw -q compile
printf 'h\ns\n' | java -cp target/classes blackjack.BlackjackApp 194 > before.txt

# （切り出したあと）
./mvnw -q compile
printf 'h\ns\n' | java -cp target/classes blackjack.BlackjackApp 194 > after.txt
diff before.txt after.txt
```

ポイント	理由
引数でシード（194）を渡す	山札の切り方が、毎回同じになる
printf で入力を渡す	Hit・Stand の操作が、毎回同じになる
diff で何も表示されない	切り出しの前後で、動きが変わっていない

※ シード（seed）：乱数（ランダムな値）を作るときの「元になる数」です。同じシードを渡すと、毎回同じ順番で同じ乱数が出てきます。

03

課題

書いて、分けて、確かめる

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題1

ルールの中にある法則を見つけよう

3〜4人で以下を話し合いましょう。

1. 手札の点数は、どう数えればよいか。A が2枚以上あるときはどうなるか

2. プレイヤーとディーラーで、同じところと違うところは何か

3. 「勝ち・負け・引き分け」は、何をどの順に確かめれば決まるか

4. ルールに出てくる「もの・名詞」は何か。それぞれが「すること・動詞」は何か
   （それぞれ、クラス・フィールド・メソッドの候補になることをイメージしながら考えてください）

ヒント：4は、ルールの説明に出てくる名詞と動詞を拾い出すところから始めます。カード、山札、手札……と、思いつくだけ挙げてみましょう。

検討例 ・ 課題1

点数・参加者・勝敗・もの を、言葉で言い表す

1. 手札の点数は、どう数えればよいか。A が2枚以上あるときはどうなるか

→ A をすべて 1 点で数え、10 を足しても 21 以下なら足す。11 点にできる A は1枚だけ

2. プレイヤーとディーラーで、同じところと違うところは何か

→ 手札を持ち、引き、点数を数えるのは同じ。もう1枚引くかの決め方だけが違う

3. 「勝ち・負け・引き分け」は、何をどの順に確かめれば決まるか

→ あなたのバースト → ディーラーのバースト → 点数の比べ合い、の順

4. ルールに出てくる「もの・名詞」は何か。それぞれが「すること・動詞」は何か
（それぞれ、クラス・フィールド・メソッドの候補になることをイメージしながら考えてください）

→ カード（点数を持つ）、山札（1枚引かれる）、手札（点数を数える）、参加者（引くか決める）

検討例 ・ 課題1

見つけた法則が、分け方の手がかりになる

対象	見つけた法則	分けるときに考えること
点数	A は 1 点で数え、条件つきで 10 足す	点数を数える処理を、1か所にまとめる
手札	点数はカードが変わるたびに変わる	点数を別に覚えず、いつもカードから数える
参加者	同じ動きで、引くかの決め方だけ違う	同じ流れは共通にし、決め方だけを任せる
勝敗	バーストを先に見て、次に点数を比べる	確かめる順番を、1か所にまとめる
もの	カード・山札・手札・参加者	名詞はクラスの、動詞はメソッドの候補

いきなりクラスを決めなくてよい。まず main に書き、ここで見つけた「もの」を思い出しながら分けていく。

Maven Wrapper を取得する

Maven Wrapper を直接ダウンロードし、使う Maven のバージョンを指定します。

```
WRAPPER_VERSION=3.3.4
MAVEN_VERSION=3.10.0

mkdir blackjack && cd blackjack

curl -fsSLO "https://repo.maven.apache.org/maven2/org/apache/maven/wrapper/maven-wrapper-
distribution/${WRAPPER_VERSION}/maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip"
unzip -l maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip
unzip maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip
rm maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip

mkdir -p .mvn/wrapper
cat > .mvn/wrapper/maven-wrapper.properties <<EOF
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/${MAVEN_VERSION}/apache-
maven-${MAVEN_VERSION}-bin.zip
EOF

chmod +x mvnw
```

準備 ・ プロジェクトを用意する（2/3）

Maven Wrapper でひな形を作る

取得した mvnw で、Maven 公式のテンプレート（archetype）からプロジェクトを生成します。blackjack フォルダの中で実行してください。

```
./mvnw archetype:generate \
  -DarchetypeGroupId=org.apache.maven.archetypes \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.5 \
  -DjavaCompilerVersion=25 \
  -DgroupId=com.example \
  -DartifactId=blackjack \
  -Dpackage=blackjack \
  -DinteractiveMode=false

mv blackjack/* ./ && rm -rf blackjack/
```

archetype:generate は、artifactId と同じ名前（blackjack）のフォルダを作って、その中にプロジェクトを生成します。最後の行で、生成された中身を blackjack の直下に移しています。

-DjavaCompilerVersion=25 で、使う Java のバージョンを 25 に指定しています（指定しないと 17 になります）。あらかじめ JDK 25 をインストールしておいてください。

準備 ・ プロジェクトを用意する（3/3）

.gitignore を作る

ビルド結果や IDE の設定ファイルを、Git の管理から外します。blackjack フォルダの中で実行してください。

```
cat > .gitignore <<'EOF'
target/
.idea/
*.iml
.vscode/
.DS_Store
mvnwDebug*
before.txt
after.txt
EOF
```

できあがる構成

```
blackjack/
├── .gitignore
├── .mvn/wrapper/
│   └── maven-wrapper.properties
├── mvnw
├── mvnw.cmd
├── pom.xml
└── src/
    ├── main/java/blackjack/
    └── test/java/blackjack/
```

.mvn フォルダは Git の管理から外さないでください。使う Maven のバージョンが書かれています。

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題2

main に書き下す

BlackjackApp クラスの main メソッドの中に、ゲームのすべてを書いて動かします。まずは動けばよく、きれいに書く必要はありません。

1. 新しいクラスもメソッドも作らず、main 1つの中に書く
2. カードは int（0〜51 の番号）や String で表してよい
3. 実行時の引数でシードを受け取り、new Random(シード) と Collections.shuffle で山札を切る
4. 画面の表示は、次のページの表示例に合わせる

完成したら、課題3以降で比べるために before.txt を作っておく（進め方「動きを変えずに分ける」のコマンド）

演習 ・ 課題2

画面の表示例

画面の表示はすべて英語にします。カードは「7 of Hearts」のように、ランクとスートで表示します。

```
$ ./mvnw -q compile
$ java -cp target/classes blackjack.BlackjackApp 194
Dealer: 10 of Spades, [hidden]
You: J of Hearts, 8 of Hearts (18)
[h]it or [s]tand? > h
You drew A of Spades.
You: J of Hearts, 8 of Hearts, A of Spades (19)
[h]it or [s]tand? > s
Dealer: 10 of Spades, 4 of Hearts (14)
Dealer drew 4 of Diamonds.
Dealer: 10 of Spades, 4 of Hearts, 4 of Diamonds (18)

You: 19 / Dealer: 18
You win!
```

バーストしたら「Bust!」と表示します。最後に両者の点数と、You win! ・ You lose... ・ Push. のどれかを表示します。

検討例 ・ 課題2

main の中身を、かたまりで見る

書き上げた main を上から読むと、次のかたまりが並んでいます。

かたまり	やっていること
山札を作る	0〜51 の番号を並べ、シードを使って切る
配る	あなたとディーラーに2枚ずつ配り、ディーラーの1枚目を表示する
あなたの番	点数を数える・手札を表示する・h か s を受け取る・1枚引く
ディーラーの番	点数を数える・手札を表示する・17 点未満なら1枚引く
勝敗	両者の点数を表示し、バーストと点数で勝敗を決める

番号 c のカードは、c % 13 でランク（0 が A）、c / 13 でスートがわかる。点数は Math.min(c % 13 + 1, 10)

解答例 ・ 課題2（1/3）

main に書き下した例

BlackjackApp.java

```
public class BlackjackApp {
    public static void main(String[] args) {
        long seed = args.length > 0
                ? Long.parseLong(args[0]) : System.nanoTime();
        Random random = new Random(seed);
        Scanner scanner = new Scanner(System.in);
        String[] ranks = {"A", "2", "3", "4", "5", "6", "7",
                "8", "9", "10", "J", "Q", "K"};
        String[] suits = {"Spades", "Hearts", "Diamonds", "Clubs"};

        // 山札：0〜51 の番号で 52 枚を表し、よく切る
        List<Integer> deck = new ArrayList<>();
        for (int i = 0; i < 52; i++) {
            deck.add(i);
        }
        Collections.shuffle(deck, random);

        // 2 枚ずつ配る
        List<Integer> player = new ArrayList<>();
        List<Integer> dealer = new ArrayList<>();
        for (int i = 0; i < 2; i++) {
            player.add(deck.remove(0));
            dealer.add(deck.remove(0));
        }
        int up = dealer.get(0);
```

BlackjackApp.java（続き）

```
        System.out.println("Dealer: " + ranks[up % 13] + " of "
                + suits[up / 13] + ", [hidden]");

        // プレイヤーの番
        int playerScore;
        while (true) {
            playerScore = 0;
            boolean hasAce = false;
            String text = "";
            for (int c : player) {
                playerScore += Math.min(c % 13 + 1, 10);
                if (c % 13 == 0) {
                    hasAce = true;
                }
                if (!text.isEmpty()) {
                    text += ", ";
                }
                text += ranks[c % 13] + " of " + suits[c / 13];
            }
            if (hasAce && playerScore + 10 <= 21) {
                playerScore += 10;
            }
            System.out.println("You: " + text
                    + " (" + playerScore + ")");
            if (playerScore > 21) {
```

解答例 ・ 課題2（2/3）

main に書き下した例

BlackjackApp.java（続き）

```
                System.out.println("Bust!");
                break;
            }
            String input = "";
            while (!input.equals("h") && !input.equals("s")) {
                System.out.print("[h]it or [s]tand? > ");
                input = scanner.nextLine().trim();
            }
            if (input.equals("s")) {
                break;
            }
            int c = deck.remove(0);
            player.add(c);
            System.out.println("You drew " + ranks[c % 13]
                    + " of " + suits[c / 13] + ".");
        }

        // ディーラーの番（プレイヤーがバーストしていなければ）
        int dealerScore;
        while (true) {
            dealerScore = 0;
            boolean hasAce = false;
            String text = "";
            for (int c : dealer) {
                dealerScore += Math.min(c % 13 + 1, 10);
```

BlackjackApp.java（続き）

```
                if (c % 13 == 0) {
                    hasAce = true;
                }
                if (!text.isEmpty()) {
                    text += ", ";
                }
                text += ranks[c % 13] + " of " + suits[c / 13];
            }
            if (hasAce && dealerScore + 10 <= 21) {
                dealerScore += 10;
            }
            if (playerScore > 21) {
                break;
            }
            System.out.println("Dealer: " + text
                    + " (" + dealerScore + ")");
            if (dealerScore > 21) {
                System.out.println("Bust!");
                break;
            }
            if (dealerScore >= 17) {
                break;
            }
            int c = deck.remove(0);
            dealer.add(c);
```

解答例 ・ 課題2（3/3）

main に書き下した例

BlackjackApp.java（続き）

```
            System.out.println("Dealer drew " + ranks[c % 13]
                    + " of " + suits[c / 13] + ".");
        }

        // 勝敗
        System.out.println();
        System.out.println("You: " + playerScore
                + " / Dealer: " + dealerScore);
        if (playerScore > 21) {
            System.out.println("You lose...");
        } else if (dealerScore > 21) {
            System.out.println("You win!");
        } else if (playerScore > dealerScore) {
            System.out.println("You win!");
        } else if (playerScore < dealerScore) {
            System.out.println("You lose...");
        } else {
            System.out.println("Push.");
        }
    }
}
```

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題3

メソッドに切り出す

課題2の main から、ひとまとまりの処理を static メソッドとして切り出します。

1. main を読み、ひとまとまりの処理に「〜する」と名前をつける
2. 同じ処理が2か所にあれば、1つのメソッドにまとめる
3. 1つ切り出すごとにコンパイルして動かし、diff before.txt after.txt で表示が変わらないことを確かめる

条件：main を読むだけで、ゲームの流れ（配る → あなたの番 → ディーラーの番 → 勝敗）がわかるようにする

検討例 ・ 課題3

メソッドに切り出す

役割	作るもの
引数からシードを決める	seedFrom(String[] args): long
52 枚をそろえて切る	newShuffledDeck(Random random): List<Integer>
山札から1枚引く	draw(List<Integer> deck): int
カードの表示名を作る（例：7 of Hearts）	label(int card): String
手札の点数を数える	score(List<Integer> hand): int
バーストしているかを検査	isBust(List<Integer> hand): boolean
手札の表示を作る（例：…(18)）	handLabel(List<Integer> hand): String
あなたの番を進める	playerTurn(deck, hand, scanner): void
h か s を受け取る	askHit(Scanner scanner): boolean
ディーラーの番を進める	dealerTurn(deck, hand): void
勝敗を表示する	printResult(player, dealer): void

検討例 ・ 課題3 クラス図

同じデータを受け取るメソッドが集まっている

課題3のメソッドを、受け取るデータで分けてみます。いつも同じデータを受け取るメソッドは、そのデータと一緒にクラスにまとめる候補です。

受け取るデータ	メソッド	次の課題では
List<Integer> deck	newShuffledDeck(), draw()	山札（Deck）にまとめる
int card	label()	カード（Card）にまとめる
List<Integer> hand	score(), isBust(), handLabel()	手札（Hand）にまとめる
deck と hand の両方	playerTurn(), dealerTurn()	ゲームを進める側に残す
2人分の hand	printResult()	ゲームを進める側に残す

「どのデータを受け取るか」が、クラスの境目を教えてくれる。

解答例 ・ 課題3（1/3）

メソッドに切り出した例

BlackjackApp.java

```
public class BlackjackApp {
    private static final String[] RANKS = {"A", "2", "3", "4",
            "5", "6", "7", "8", "9", "10", "J", "Q", "K"};
    private static final String[] SUITS =
            {"Spades", "Hearts", "Diamonds", "Clubs"};

    public static void main(String[] args) {
        Random random = new Random(seedFrom(args));
        Scanner scanner = new Scanner(System.in);

        List<Integer> deck = newShuffledDeck(random);
        List<Integer> player = new ArrayList<>();
        List<Integer> dealer = new ArrayList<>();
        for (int i = 0; i < 2; i++) {
            player.add(draw(deck));
            dealer.add(draw(deck));
        }
        System.out.println("Dealer: "
                + label(dealer.get(0)) + ", [hidden]");

        playerTurn(deck, player, scanner);
        if (!isBust(player)) {
            dealerTurn(deck, dealer);
        }
        printResult(player, dealer);
```

BlackjackApp.java（続き）

```
    }

    static long seedFrom(String[] args) {
        return args.length > 0
                ? Long.parseLong(args[0]) : System.nanoTime();
    }

    static List<Integer> newShuffledDeck(Random random) {
        List<Integer> deck = new ArrayList<>();
        for (int i = 0; i < 52; i++) {
            deck.add(i);
        }
        Collections.shuffle(deck, random);
        return deck;
    }

    static int draw(List<Integer> deck) {
        return deck.remove(0);
    }

    static String label(int card) {
        return RANKS[card % 13] + " of " + SUITS[card / 13];
    }

    static int score(List<Integer> hand) {
```

解答例 ・ 課題3（2/3）

メソッドに切り出した例

BlackjackApp.java（続き）

```
        int score = 0;
        boolean hasAce = false;
        for (int card : hand) {
            score += Math.min(card % 13 + 1, 10);
            if (card % 13 == 0) {
                hasAce = true;
            }
        }
        if (hasAce && score + 10 <= 21) {
            score += 10;
        }
        return score;
    }

    static boolean isBust(List<Integer> hand) {
        return score(hand) > 21;
    }

    static String handLabel(List<Integer> hand) {
        List<String> labels = new ArrayList<>();
        for (int card : hand) {
            labels.add(label(card));
        }
        return String.join(", ", labels)
                + " (" + score(hand) + ")";
```

BlackjackApp.java（続き）

```
    }

    static void playerTurn(List<Integer> deck,
            List<Integer> hand, Scanner scanner) {
        System.out.println("You: " + handLabel(hand));
        while (!isBust(hand) && askHit(scanner)) {
            int card = draw(deck);
            hand.add(card);
            System.out.println("You drew " + label(card) + ".");
            System.out.println("You: " + handLabel(hand));
        }
        if (isBust(hand)) {
            System.out.println("Bust!");
        }
    }

    static boolean askHit(Scanner scanner) {
        String input = "";
        while (!input.equals("h") && !input.equals("s")) {
            System.out.print("[h]it or [s]tand? > ");
            input = scanner.nextLine().trim();
        }
        return input.equals("h");
    }
```

解答例 ・ 課題3（3/3）

メソッドに切り出した例

BlackjackApp.java（続き）

```
    static void dealerTurn(List<Integer> deck,
            List<Integer> hand) {
        System.out.println("Dealer: " + handLabel(hand));
        while (score(hand) < 17) {
            int card = draw(deck);
            hand.add(card);
            System.out.println("Dealer drew " + label(card) + ".");
            System.out.println("Dealer: " + handLabel(hand));
        }
        if (isBust(hand)) {
            System.out.println("Bust!");
        }
    }

    static void printResult(List<Integer> player,
            List<Integer> dealer) {
        int playerScore = score(player);
        int dealerScore = score(dealer);
        System.out.println();
        System.out.println("You: " + playerScore
                + " / Dealer: " + dealerScore);
        if (isBust(player)) {
            System.out.println("You lose...");
        } else if (isBust(dealer)) {
            System.out.println("You win!");
```

BlackjackApp.java（続き）

```
        } else if (playerScore > dealerScore) {
            System.out.println("You win!");
        } else if (playerScore < dealerScore) {
            System.out.println("You lose...");
        } else {
            System.out.println("Push.");
        }
    }
}
```

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題4

クラスに切り出す

課題3で、いつも同じデータを受け取っていたメソッドを、そのデータと一緒にクラスへまとめます。int のカードも、ランクとスートを持つものに置き換えます。

1. フィールドはすべて private にし、手札や山札の List を外から直接変更できないようにする
2. カードは、ランクとスートが必ずそろっている
3. 山札から同じカードは2度出ず、空の山札からは引けない
4. 手札の点数は、いつもルールどおりに数えられる

ヒント：山札は課題3と同じ順（スートごとに A, 2, …, K）に並べてから切ると、同じシードで同じ表示になる

検討例 ・ 課題4

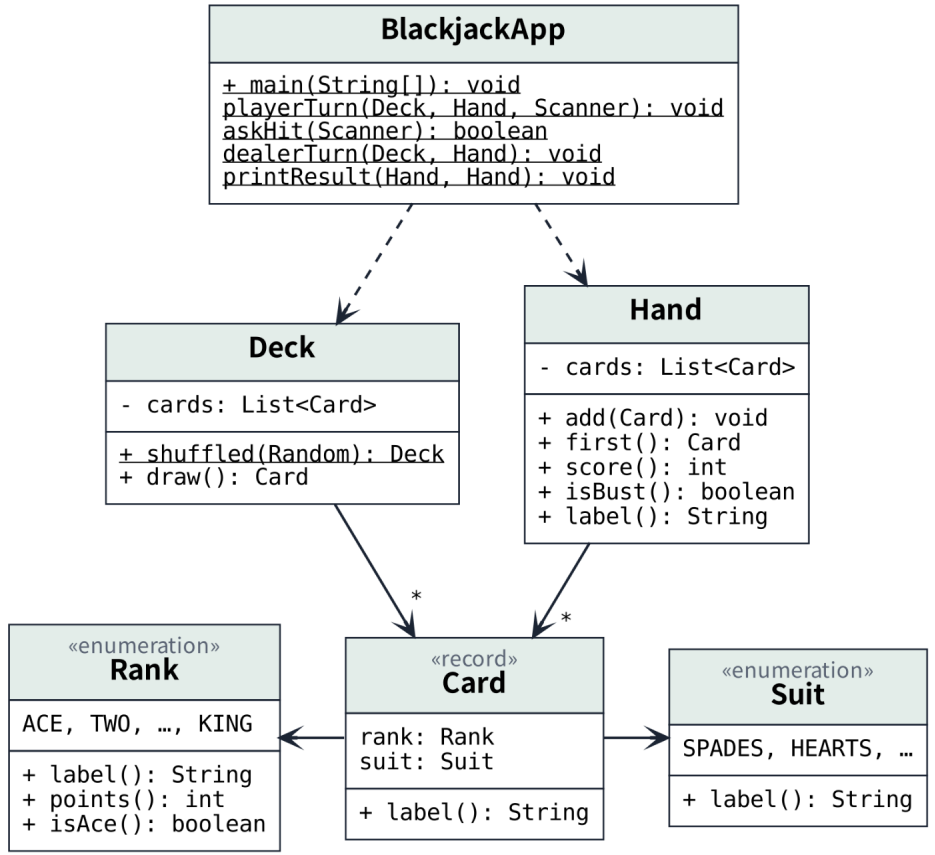
クラスに切り出す

役割	作るもの
13種類のランク（A〜K）と、それぞれの点数	enum Rank：label(), points(), isAce()
4種類のスート	enum Suit：label()
ランクとスートを組で持つ（どちらも欠けてはいけない）	record Card(Rank rank, Suit suit)：label()
52 枚をそろえて切った山札を作る	Deck：static shuffled(Random): Deck
山札から1枚引く（空なら引けない）	Deck：draw(): Card
手札にカードを加える・1枚目を見る	Hand：add(Card), first(): Card
手札の点数を数える・バーストを検査	Hand：score(): int, isBust(): boolean
手札の表示を作る	Hand：label(): String

BlackjackApp には、あなたの番・ディーラーの番・勝敗の表示が残る

検討例 ・ 課題4 クラス図

データと処理を、まとめて閉じ込める



«record» Card

ランクとスートを組で持ち、どちらも欠けない

Deck

カードの List を private で持ち、draw() でしか減らせない

Hand

点数を持たず、score() で毎回カードから数える

破線の矢印

BlackjackApp は Deck と Hand を受け取って使う

解答例 ・ 課題4（1/5）

クラスに切り出した例

Rank.java

```
/** カードのランク（数字）と、その点数。A は 1 点として持つ。 */
public enum Rank {
    ACE("A", 1),
    TWO("2", 2),
    THREE("3", 3),
    FOUR("4", 4),
    FIVE("5", 5),
    SIX("6", 6),
    SEVEN("7", 7),
    EIGHT("8", 8),
    NINE("9", 9),
    TEN("10", 10),
    JACK("J", 10),
    QUEEN("Q", 10),
    KING("K", 10);

    private final String label;
    private final int points;

    Rank(String label, int points) {
        this.label = label;
        this.points = points;
    }

    public String label() {
```

Rank.java（続き）

```
        return label;
    }

    public int points() {
        return points;
    }

    public boolean isAce() {
        return this == ACE;
    }
}
```

Suit.java

```
/** カードのスート（マーク）。点数には関係しない。 */
public enum Suit {
    SPADES("Spades"),
    HEARTS("Hearts"),
    DIAMONDS("Diamonds"),
    CLUBS("Clubs");

    private final String label;

    Suit(String label) {
        this.label = label;
    }
```

解答例 ・ 課題4（2/5）

クラスに切り出した例

Suit.java（続き）

```
    public String label() {
        return label;
    }
}
```

Card.java

```
/** ランクとスートの組。どちらも欠けてはいけない。 */
public record Card(Rank rank, Suit suit) {
    public Card {
        Objects.requireNonNull(rank);
        Objects.requireNonNull(suit);
    }

    public String label() {
        return rank.label() + " of " + suit.label();
    }
}
```

Deck.java

```
/** 山札。同じカードは入らず、空の山札からは引けない。 */
public class Deck {
    private final List<Card> cards;

    private Deck(List<Card> cards) {
        this.cards = cards;
    }

    /** 52 枚をそろえ、よく切った山札を作る */
    public static Deck shuffled(Random random) {
        List<Card> cards = new ArrayList<>();
        for (Suit suit : Suit.values()) {
            for (Rank rank : Rank.values()) {
                cards.add(new Card(rank, suit));
            }
        }
        Collections.shuffle(cards, random);
        return new Deck(cards);
    }

    /** 一番上の 1 枚を引く */
    public Card draw() {
        if (cards.isEmpty()) {
            throw new IllegalStateException("Deck is empty");
        }
```

解答例 ・ 課題4（3/5）

クラスに切り出した例

Deck.java（続き）

```
        return cards.remove(0);
    }
}
```

Hand.java

```
/** 手札。点数は持たず、いつもカードから数え直す。 */
public class Hand {
    private final List<Card> cards = new ArrayList<>();

    public void add(Card card) {
        cards.add(card);
    }

    public Card first() {
        return cards.get(0);
    }

    /** A は 1 点で数え、21 を超えなければ 1 枚だけ 11 点にする */
    public int score() {
        int score = 0;
        boolean hasAce = false;
        for (Card card : cards) {
            score += card.rank().points();
            if (card.rank().isAce()) {
                hasAce = true;
```

Hand.java（続き）

```
                }
            }
            if (hasAce && score + 10 <= 21) {
                score += 10;
            }
            return score;
    }

    public boolean isBust() {
        return score() > 21;
    }

    public String label() {
        List<String> labels = new ArrayList<>();
        for (Card card : cards) {
            labels.add(card.label());
        }
        return String.join(", ", labels) + " (" + score() + ")";
    }
}
```

解答例 ・ 課題4（4/5）

クラスに切り出した例

BlackjackApp.java

```
public class BlackjackApp {
    public static void main(String[] args) {
        long seed = args.length > 0
                ? Long.parseLong(args[0]) : System.nanoTime();
        Deck deck = Deck.shuffled(new Random(seed));
        Scanner scanner = new Scanner(System.in);

        Hand player = new Hand();
        Hand dealer = new Hand();
        for (int i = 0; i < 2; i++) {
            player.add(deck.draw());
            dealer.add(deck.draw());
        }
        System.out.println("Dealer: "
                + dealer.first().label() + ", [hidden]");

        playerTurn(deck, player, scanner);
        if (!player.isBust()) {
            dealerTurn(deck, dealer);
        }
        printResult(player, dealer);
    }

    static void playerTurn(Deck deck, Hand hand,
            Scanner scanner) {
```

BlackjackApp.java（続き）

```
        System.out.println("You: " + hand.label());
        while (!hand.isBust() && askHit(scanner)) {
            Card card = deck.draw();
            hand.add(card);
            System.out.println("You drew " + card.label() + ".");
            System.out.println("You: " + hand.label());
        }
        if (hand.isBust()) {
            System.out.println("Bust!");
        }
    }

    static boolean askHit(Scanner scanner) {
        String input = "";
        while (!input.equals("h") && !input.equals("s")) {
            System.out.print("[h]it or [s]tand? > ");
            input = scanner.nextLine().trim();
        }
        return input.equals("h");
    }

    static void dealerTurn(Deck deck, Hand hand) {
        System.out.println("Dealer: " + hand.label());
        while (hand.score() < 17) {
            Card card = deck.draw();
```

解答例 ・ 課題4（5/5）

クラスに切り出した例

BlackjackApp.java（続き）

```
            hand.add(card);
            System.out.println("Dealer drew " + card.label() + ".");
            System.out.println("Dealer: " + hand.label());
        }
        if (hand.isBust()) {
            System.out.println("Bust!");
        }
    }

    static void printResult(Hand player, Hand dealer) {
        System.out.println();
        System.out.println("You: " + player.score()
                + " / Dealer: " + dealer.score());
        if (player.isBust()) {
            System.out.println("You lose...");
        } else if (dealer.isBust()) {
            System.out.println("You win!");
        } else if (player.score() > dealer.score()) {
            System.out.println("You win!");
        } else if (player.score() < dealer.score()) {
            System.out.println("You lose...");
        } else {
            System.out.println("Push.");
        }
    }
```

BlackjackApp.java（続き）

```
}
```

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題5

役割で分ける

playerTurn と dealerTurn は、よく似ていました。違うところだけをそれぞれに任せ、勝敗とゲームの進行もクラスにします。

1. playerTurn と dealerTurn を並べ、同じところと違うところに印をつける
2. 違うところ（もう1枚引くか）だけを、あなたとディーラーがそれぞれ決められるようにする
3. 勝ち・負け・引き分けを表すものを作り、勝敗の決め方をそこにまとめる
4. ゲームを進めるクラスを作り、main は部品を組み立てて始めるだけにする

条件：ゲームを進めるクラスは、相手があなたかディーラーかを if で書き分けない

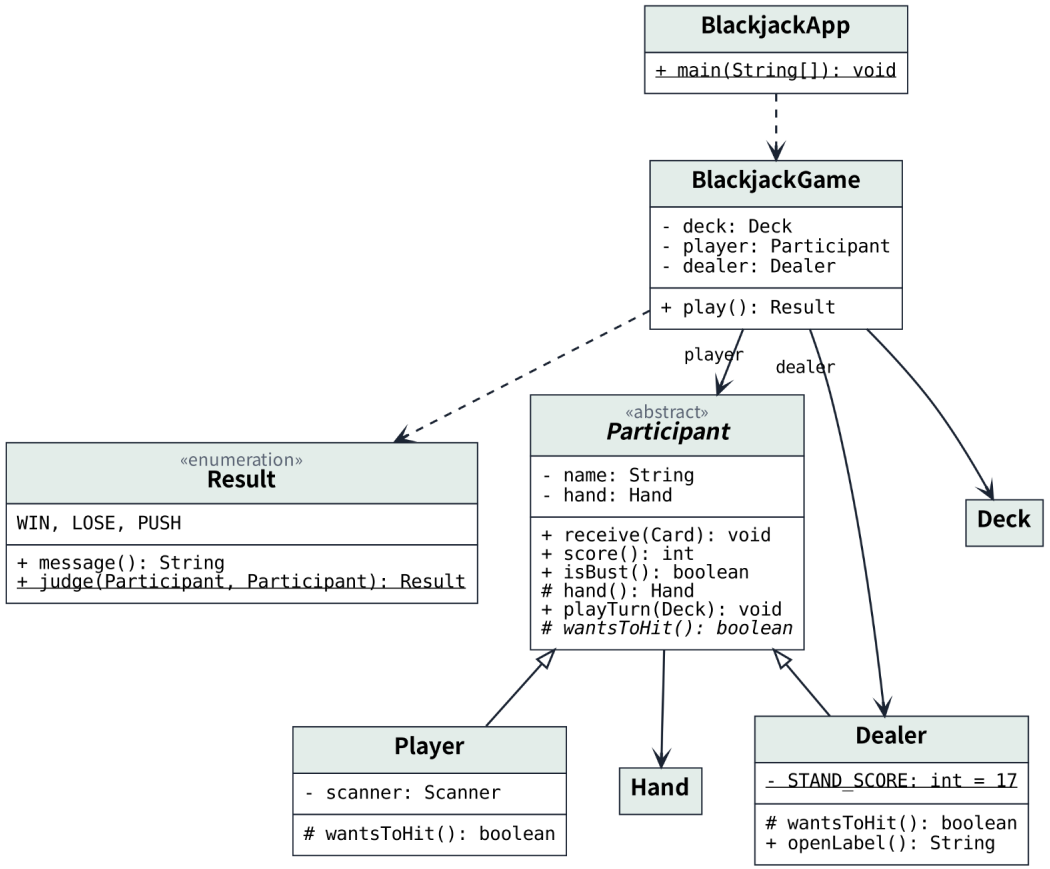
検討例 ・ 課題5

役割で分ける

役割	作るもの
手札を持ち、自分の番を進める参加者	abstract class Participant
バーストせず、引きたい間だけ引く（流れは共通）	Participant：final playTurn(Deck)
もう1枚引くか（決め方は参加者ごと）	Participant：abstract wantsToHit()
入力で引くかを決める	Player extends Participant
17 点未満なら引く・1枚目だけ見せる	Dealer extends Participant：openLabel()
勝ち・負け・引き分けと、その表示	enum Result：WIN, LOSE, PUSH, message()
バースト → 点数の順で勝敗を決める	Result：static judge(player, dealer)
配る → それぞれの番 → 勝敗 の順に進める	BlackjackGame：play(): Result

検討例 ・ 課題5 クラス図

違いは、継承したクラスに任せる



«abstract» Participant

手番の流れ playTurn() を持ち、wantsToHit() だけを任せる

白抜きの三角

Player と Dealer は Participant を継承する

BlackjackGame

参加者の種類を区別せずに playTurn() を呼ぶ

Deck・Hand

中身は課題4と同じ（図では省略）

解答例 ・ 課題5（1/4）

役割で分けた例

Participant.java

```
/** 勝負に参加する人。手札を持ち、自分の番を進める。 */
public abstract class Participant {
    private final String name;
    private final Hand hand = new Hand();

    protected Participant(String name) {
        this.name = name;
    }

    public void receive(Card card) {
        hand.add(card);
    }

    public int score() {
        return hand.score();
    }

    public boolean isBust() {
        return hand.isBust();
    }

    protected Hand hand() {
        return hand;
    }
```

Participant.java（続き）

```
    /** バーストせず、引きたい間だけ山札から引く */
    public final void playTurn(Deck deck) {
        System.out.println(name + ": " + hand.label());
        while (!hand.isBust() && wantsToHit()) {
            Card card = deck.draw();
            hand.add(card);
            System.out.println(name + " drew " + card.label() + ".");
            System.out.println(name + ": " + hand.label());
        }
        if (hand.isBust()) {
            System.out.println("Bust!");
        }
    }

    /** もう 1 枚引くかどうか。決め方は参加者ごとに違う */
    protected abstract boolean wantsToHit();
}
```

解答例 ・ 課題5（2/4）

役割で分けた例

Player.java

```
/** 遊ぶ人。引くかどうかを、入力で決める。 */
public class Player extends Participant {
    private final Scanner scanner;

    public Player(Scanner scanner) {
        super("You");
        this.scanner = scanner;
    }

    @Override
    protected boolean wantsToHit() {
        String input = "";
        while (!input.equals("h") && !input.equals("s")) {
            System.out.print("[h]it or [s]tand? > ");
            input = scanner.nextLine().trim();
        }
        return input.equals("h");
    }
}
```

Dealer.java

```
/** ディーラー。17 点以上になるまで、必ず引く。 */
public class Dealer extends Participant {
    private static final int STAND_SCORE = 17;

    public Dealer() {
        super("Dealer");
    }

    @Override
    protected boolean wantsToHit() {
        return score() < STAND_SCORE;
    }

    /** 1 枚目だけを見せる */
    public String openLabel() {
        return hand().first().label() + ", [hidden]";
    }
}
```

解答例 ・ 課題5（3/4）

役割で分けた例

Result.java

```
/** プレイヤーから見た勝負の結果 */
public enum Result {
    WIN("You win!"),
    LOSE("You lose..."),
    PUSH("Push.");

    private final String message;

    Result(String message) {
        this.message = message;
    }

    public String message() {
        return message;
    }

    /** バーストを先に見て、どちらもしていなければ点数で比べる */
    public static Result judge(Participant player,
            Participant dealer) {
        if (player.isBust()) {
            return LOSE;
        }
        if (dealer.isBust()) {
            return WIN;
        }
```

Result.java（続き）

```
        int diff = Integer.compare(player.score(), dealer.score());
        if (diff > 0) {
            return WIN;
        }
        return diff < 0 ? LOSE : PUSH;
    }
}
```

BlackjackGame.java

```
/** 1 回の勝負を、配る・それぞれの番・勝敗の順に進める */
public class BlackjackGame {
    private final Deck deck;
    private final Participant player;
    private final Dealer dealer;

    public BlackjackGame(Deck deck, Participant player,
            Dealer dealer) {
        this.deck = deck;
        this.player = player;
        this.dealer = dealer;
    }

    public Result play() {
        for (int i = 0; i < 2; i++) {
            player.receive(deck.draw());
```

解答例 ・ 課題5（4/4）

役割で分けた例

BlackjackGame.java（続き）

```
            dealer.receive(deck.draw());
        }
        System.out.println("Dealer: " + dealer.openLabel());

        player.playTurn(deck);
        if (!player.isBust()) {
            dealer.playTurn(deck);
        }

        Result result = Result.judge(player, dealer);
        System.out.println();
        System.out.println("You: " + player.score()
                + " / Dealer: " + dealer.score());
        System.out.println(result.message());
        return result;
    }
}
```

BlackjackApp.java

```
public class BlackjackApp {
    public static void main(String[] args) {
        long seed = args.length > 0
                ? Long.parseLong(args[0]) : System.nanoTime();
        Deck deck = Deck.shuffled(new Random(seed));
        Player player = new Player(new Scanner(System.in));

        new BlackjackGame(deck, player, new Dealer()).play();
    }
}
```

課題

実装で意識すること

プラクティス	このゲームでの例
責務の分割 1つのクラスは1つのことを知る	点数の数え方は Hand、引くかの決め方は Player と Dealer 勝敗の決め方は Result、進める順番は BlackjackGame が知っている
カプセル化 不変条件を守る	カードはランクとスートが必ずそろい、山札から同じカードは2度出ない 手札の点数は持たずに、いつもカードから数え直す
多態性 種類を知らずに任せる	BlackjackGame は、あなたとディーラーを区別せずに playTurn() を呼ぶ

データとそれを扱う処理は一緒に置き、種類ごとの違いはそれぞれに任せる。

課題 ・ ふりかえり

最初の main と比べてみよう

課題2と課題5のコードを並べて、次のことを話し合いましょう。

1. 「ディーラーは 18 点以上になるまで引く」に変えるとき、それぞれどこを直すか
2. 点数の数え方が正しいかを確かめたいとき、それぞれどうやって確かめるか
3. 課題4・5で分けた境目は、課題1で見つけた「もの」と同じだったか

分けた境目が、変更とテストのしやすさにつながっている。

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 発展課題

仕様を変えてみる

早く終わった人は、次の変更に挑戦しましょう。変更したら、直したクラスとメソッドの数を数えてください。

1. 最初の2枚で 21 点（ブラックジャック）なら、ほかの 21 点より強いことにする
2. 決まった作戦で自動で遊ぶプレイヤーを作り、100回遊ばせて勝率を出す
3. チップを賭けられるようにし、勝てば2倍、引き分けなら賭けたぶんが戻るようにする

直す箇所が少なく、1か所に集まっているほど、変更に強い設計です。BlackjackGame を直す必要があったかどうかにも注目しましょう。

演習 ・ 付録課題

手札の点数をテストする

Hand を切り出したので、ゲームを動かさなくても点数の数え方を確かめられます。src/test/java/blackjack/HandTest.java に JUnit 5 のテストを書き、./mvnw -q test で実行しましょう。

手札	期待する点数
K, 7	17
A, K	21
A, A	12
A, A, 9	21
A, 6, K	17
K, Q, 2	22（バースト）

どのケースが A の数え方の法則を確かめているかを、テストの名前で言い表してみましょう。

解答例 ・ 付録課題

Hand のテストの例

HandTest.java

```
class HandTest {
    private static Hand handOf(Rank... ranks) {
        Hand hand = new Hand();
        for (Rank rank : ranks) {
            hand.add(new Card(rank, Suit.SPADES));
        }
        return hand;
    }

    @Test
    @DisplayName("絵札は 10 点で数える")
    void faceCardCountsAsTen() {
        assertEquals(17, handOf(Rank.KING, Rank.SEVEN).score());
    }

    @Test
    @DisplayName("A を 11 点と数えて 21 点になる")
    void aceCountsAsEleven() {
        assertEquals(21, handOf(Rank.ACE, Rank.KING).score());
    }

    @Test
    @DisplayName("11 点にできる A は 1 枚だけ")
    void onlyOneAceCountsAsEleven() {
        assertEquals(12, handOf(Rank.ACE, Rank.ACE).score());
```

HandTest.java（続き）

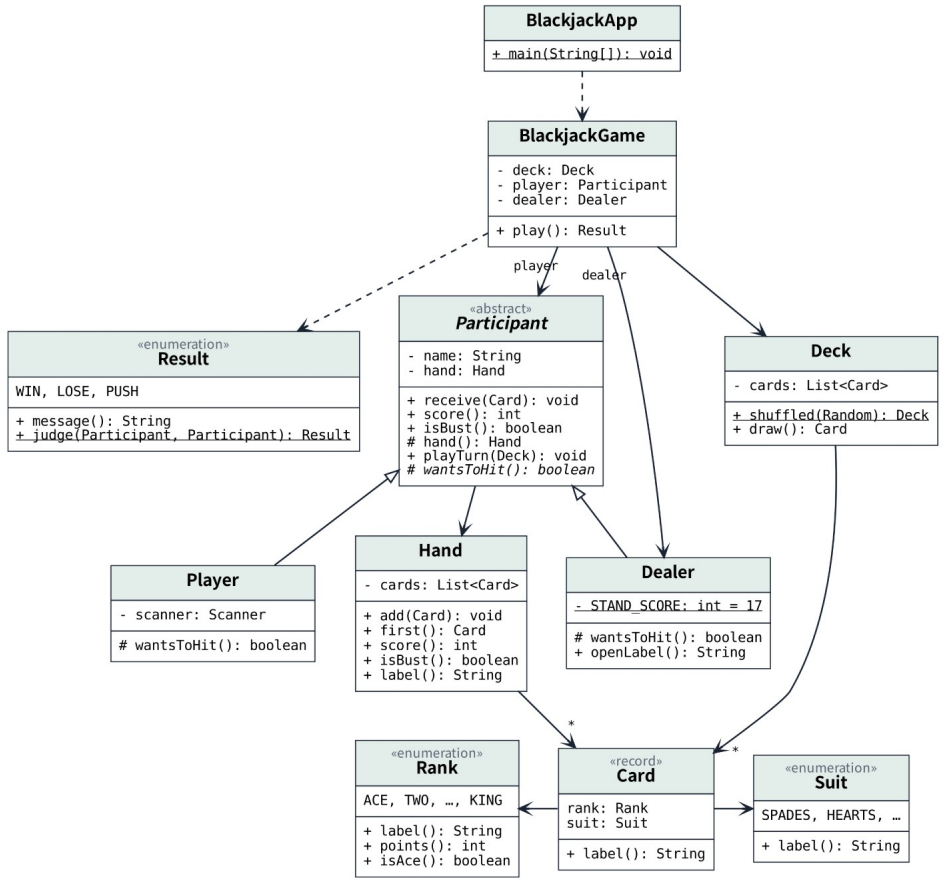
```
        assertEquals(21,
                handOf(Rank.ACE, Rank.ACE, Rank.NINE).score());
    }

    @Test
    @DisplayName("21 を超えそうなら A は 1 点になる")
    void aceCountsAsOneToAvoidBust() {
        assertEquals(17,
                handOf(Rank.ACE, Rank.SIX, Rank.KING).score());
    }

    @Test
    @DisplayName("21 を超えたらバースト")
    void overTwentyOneIsBust() {
        Hand hand = handOf(Rank.KING, Rank.QUEEN, Rank.TWO);
        assertEquals(22, hand.score());
        assertTrue(hand.isBust());
    }
}
```

検討例 ・ 付録 クラス図

全体のつながり



BlackjackApp・BlackjackGame

部品を組み立て、配る → それぞれの番 → 勝敗の順に進める

Participant・Player・Dealer

手番の流れは共通にし、引くかの決め方だけを任せる（課題5）

Deck・Hand・Card

データと処理をまとめて閉じ込める（課題4）

Rank・Suit・Result

種類と、種類ごとの値や表示を enum で表す